ONEDRAFT

CAD-AI — Aria Powered

Infrastructure & Deployment Architecture

Version 1.0

Status: Implementation Specification
Infrastructure: Containerized Cloud / Hybrid Deployment
Primary Runtime: Linux / Docker
Database: PostgreSQL
Queue: Distributed Job Queue
Object Storage: S3-Compatible Artifact Storage
Deployment Model: Environment-Separated
Architecture: Service-Oriented / Worker-Based

---

1. INFRASTRUCTURE DESIGN PRINCIPLE

OneDraft infrastructure shall provide the physical and virtual execution environment required to operate the complete OneDraft computational platform.

The infrastructure layer shall support:

API services

Domain services

Aria orchestration

Geometry computation

Regulatory analysis

Validation

Documentation generation

Background jobs

Artifact storage

PostgreSQL persistence

Event processing

Observability

The infrastructure must remain separated from the domain model.

The application defines what OneDraft does.

The infrastructure defines where and how OneDraft executes.

---

2. DEPLOYMENT ARCHITECTURE

The initial deployment topology shall be:

```
CLIENT
  ↓
EDGE / LOAD BALANCER
  ↓
API GATEWAY
  ↓
APPLICATION SERVICES
  ↓
DOMAIN SERVICES
  ↓
POSTGRESQL
  ↓
EVENT BUS
  ↓
JOB QUEUE
  ↓
WORKERS
  ↓
ARTIFACT STORAGE
```

Supporting systems:

```
OBSERVABILITY
CONFIGURATION
SECRETS
BACKUP
SECURITY
CI/CD
```

---

3. ENVIRONMENT SEPARATION

OneDraft shall maintain independent environments.

Initial environments:

```
DEVELOPMENT
TEST
STAGING
PRODUCTION
```

Each environment shall have independently controlled:

```
DATABASE
CACHE
QUEUE
OBJECT STORAGE
SECRETS
CONFIGURATION
SERVICE INSTANCES
LOGGING
MONITORING
```

Production resources must never be unintentionally shared with development resources.

---

4. CONTAINERIZATION

OneDraft application services shall be containerized.

Initial container boundaries include:

```
api
worker
geometry-worker
compliance-worker
validation-worker
documentation-worker
aria-service
event-consumer
scheduler
```

Infrastructure dependencies may additionally include:

```
postgres
redis
object-storage
message-broker
observability
```

The exact service decomposition may evolve without changing the domain contracts.

---

5. CONTAINER IMAGE STANDARD

Each deployable service shall produce an immutable container image.

Images shall identify:

```
service
version
commit
build
timestamp
```

```
environment metadata
```

Example:

```
onedraft/api:0.1.0
onedraft/geometry-worker:0.1.0
onedraft/documentation-worker:0.1.0
```

Production deployments shall reference immutable image versions rather than floating development tags.

---

6. CONTAINER SECURITY

Containers shall run with the minimum privileges required.

Production containers should use:

```
non-root users
read-only filesystems where practical
minimal base images
restricted capabilities
resource limits
network restrictions
secret injection
```

Containers must not receive unrestricted host access.

---

7. SERVICE NETWORK

Internal services shall communicate through controlled service networking.

Conceptually:

```
PUBLIC NETWORK
      ↓
EDGE
      ↓
API
      ↓
PRIVATE SERVICE NETWORK
      ↓
DATABASE / QUEUE / STORAGE / WORKERS
```

PostgreSQL shall not be directly exposed to the public internet.

Internal worker services shall not require public network exposure.

---

8. NETWORK SEGMENTATION

Infrastructure shall separate:

Public ingress

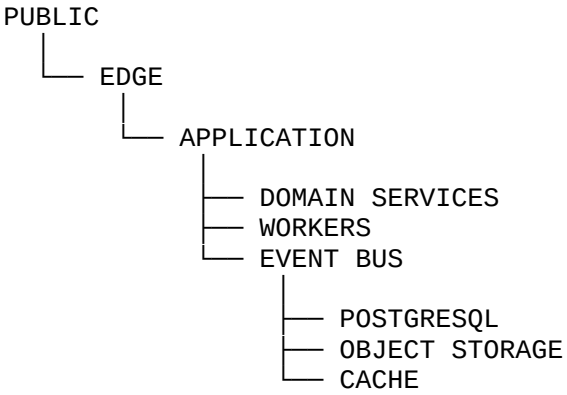
Application services

Data services

Worker services

Administrative services

The preferred topology is:



---

9. EDGE NETWORK

The edge layer shall provide:

```
TLS termination
request routing
HTTP protection
rate limiting
health routing
load balancing
request logging
```

The edge layer shall forward only valid application traffic to the API gateway.

---

10. TLS

All external API traffic shall use HTTPS.

Production endpoints shall reject unencrypted HTTP except where HTTP is explicitly required for controlled redirect behavior.

Internal encryption shall be used where infrastructure topology and deployment requirements justify it.

Certificates shall be automatically managed where supported by the deployment environment.

---

11. API SERVICE DEPLOYMENT

The API service shall be stateless.

Persistent state shall reside in:

```
PostgreSQL
object storage
event systems
job queues
approved caches
```

API instances may therefore scale horizontally.

Conceptually:

```
API INSTANCE 1
API INSTANCE 2
API INSTANCE 3
        ↓
   LOAD BALANCER
        ↓
POSTGRES / QUEUE / STORAGE
```

---

12. DOMAIN SERVICE DEPLOYMENT

Domain services shall execute application logic without requiring local persistent state.

Services shall retrieve authoritative state through controlled repositories and application interfaces.

The infrastructure must not permit service-local databases to become competing systems of record.

---

13. POSTGRESQL DEPLOYMENT

PostgreSQL is the authoritative transactional database.

Production PostgreSQL shall provide:

```
persistent storage
automated backups
```

```
replication where required
connection management
monitoring
failure recovery
migration control
```

The application shall connect through a controlled database endpoint.

---

14. DATABASE CONNECTION MANAGEMENT

Services shall use connection pooling.

The infrastructure shall enforce:

```
maximum connections
connection timeout
idle timeout
query timeout
transaction timeout
```

The system must prevent uncontrolled worker scaling from exhausting PostgreSQL connections.

---

15. DATABASE MIGRATIONS

Database migrations shall be executed through the controlled deployment process.

Migrations shall be:

```
versioned
ordered
immutable
repeatable in new environments
reviewed
tested before production
```

A production migration shall never be manually edited after deployment.

---

16. CACHE INFRASTRUCTURE

A distributed cache may be used for:

```
short-lived computation
session-adjacent data
job coordination
```

```
rate limiting
temporary locks
frequently accessed derived information
```

The cache is never the authoritative source for project state.

If cache state is lost, the system must remain reconstructable from authoritative persistence.

---

17. JOB QUEUE INFRASTRUCTURE

Long-running operations shall execute through the distributed job system.

Examples:

```
GEOMETRY_GENERATION
CODE_VALIDATION
DOCUMENT_GENERATION
DRAWING_GENERATION
PDF_EXPORT
IMPORT_PROCESSING
PACKAGE_GENERATION
```

The queue provides:

```
durability
retry
priority
concurrency control
dead-letter handling
job visibility
```

---

18. WORKER DEPLOYMENT

Workers shall be independently scalable.

Example:

```
API
 ↓
QUEUE
 ├── GEOMETRY WORKERS
 ├── VALIDATION WORKERS
 ├── DOCUMENTATION WORKERS
 ├── COMPLIANCE WORKERS
 └── IMPORT / EXPORT WORKERS
```

Worker scaling shall respond to queue depth and workload characteristics.

---

19. WORKER RESOURCE CLASSES

Different workloads require different resource profiles.

Initial classes:

```
STANDARD
CPU_INTENSIVE
MEMORY_INTENSIVE
GEOMETRY
DOCUMENT
AI
```

Geometry and rendering workloads may require substantially greater CPU or memory resources than ordinary API requests.

---

20. GPU INFRASTRUCTURE

GPU execution shall remain optional.

Where required for AI workloads:

```
GPU WORKER POOL
```

may be deployed separately from ordinary application workers.

The API and domain architecture must not depend on a specific GPU vendor.

---

21. OBJECT STORAGE

Generated and uploaded artifacts shall be stored outside PostgreSQL.

Object storage shall contain:

```
PDF
DWG
DXF
IFC
SVG
PNG
JPG
ZIP
survey files
source documents
generated drawing sets
document packages
```

PostgreSQL stores the authoritative metadata and references.

---

22. OBJECT STORAGE KEY STRUCTURE

Objects shall use deterministic project-oriented paths.

Conceptually:

```
/projects/{project_id}/
    /imports/
    /geometry/
    /drawings/
    /documents/
    /exports/
    /packages/
    /archives/
```

Revision-specific artifacts shall include the relevant revision or model version.

---

23. ARTIFACT IMMUTABILITY

Finalized artifacts shall be immutable.

A replacement artifact shall receive a new object identity or version.

The system shall not silently overwrite a finalized document package.

---

24. ARTIFACT HASHING

Generated artifacts shall receive cryptographic hashes.

At minimum:

```
SHA-256
```

The artifact record shall preserve:

```
object_reference
hash
size
content_type
created_at
project_id
revision_id
model_version_id
```

This supports document provenance and integrity verification.

---

25. EVENT BUS INFRASTRUCTURE

The event bus shall provide asynchronous communication between OneDraft subsystems.

Examples:

```
ProjectCreated
ModelChanged
RevisionCreated
CommandCompleted
ValidationCompleted
DrawingGenerated
DocumentPackageGenerated
AcceptanceRecorded
```

Events shall contain sufficient identifiers to reconstruct their context without embedding excessive mutable state.

---

26. EVENT DELIVERY

Event consumers shall assume that events may be:

```
duplicated
delayed
retried
reordered
temporarily unavailable
```

Consumers must therefore be idempotent where appropriate.

The event system must not require exactly-once delivery semantics for ordinary application correctness.

---

27. SERVICE DISCOVERY

Internal services shall be discoverable through the deployment platform.

The application should use logical service names rather than hard-coded host addresses.

Example:

```
api
postgres
queue
object-storage
geometry-worker
validation-worker
documentation-worker
```

---

28. CONFIGURATION MANAGEMENT

Runtime configuration shall be externalized.

Configuration includes:

```
database endpoints
queue endpoints
storage endpoints
service URLs
feature flags
worker limits
timeouts
logging levels
AI provider configuration
```

Configuration shall not be hard-coded into application source.

---

29. SECRETS MANAGEMENT

Secrets shall be stored separately from ordinary configuration.

Examples:

```
database passwords
API credentials
AI provider keys
storage credentials
signing keys
encryption keys
OAuth secrets
```

Secrets must never be committed to source control.

---

30. ENVIRONMENT VARIABLES

Environment variables may provide runtime configuration.

Example:

```
DATABASE_URL
REDIS_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
ARIA_PROVIDER
LOG_LEVEL
ENVIRONMENT
```

Sensitive values should be supplied through the deployment secret mechanism rather than plaintext configuration files.

---

31. RESOURCE LIMITS

Every production workload shall have defined resource boundaries.

At minimum:

```
CPU request
CPU limit
memory request
memory limit
maximum concurrency
timeout
```

Worker-specific limits shall reflect the computational profile of the workload.

---

32. AUTOSCALING

Stateless services may scale according to:

```
CPU
memory
request rate
latency
queue depth
worker utilization
```

Worker services should primarily scale according to workload demand rather than HTTP request volume.

---

33. DATABASE SCALING

PostgreSQL scaling shall proceed conservatively.

Initial strategy:

```
vertical scaling
connection pooling
query optimization
index optimization
read replicas where justified
partitioning where justified
```

Horizontal database sharding shall not be introduced until workload characteristics demonstrate a requirement.

---

34. QUEUE SCALING

Queue infrastructure shall support independent worker scaling.

Example:

```
QUEUE DEPTH ↑
      ↓
WORKER COUNT ↑
      ↓
QUEUE DEPTH ↓
```

The system shall establish maximum worker limits to prevent uncontrolled resource consumption.

---

35. STORAGE SCALING

Object storage shall scale independently from database storage.

Large artifacts must not consume PostgreSQL disk capacity.

Artifact lifecycle policies may transition inactive objects to archival storage where appropriate.

---

36. HEALTH CHECKS

Every deployable service shall expose health information.

Minimum checks:

```
LIVENESS
READINESS
DEPENDENCY HEALTH
```

A liveness failure indicates that the process requires restart or replacement.

A readiness failure indicates that the service should temporarily receive no new work.

---

37. SERVICE HEALTH ENDPOINTS

Application services shall expose controlled health endpoints.

Conceptually:

```
GET /health
GET /ready
```

Detailed infrastructure diagnostics shall not be publicly exposed.

---

38. DEPLOYMENT TOPOLOGY

A production deployment may initially consist of:

```
EDGE
API CLUSTER
DOMAIN SERVICE CLUSTER
WORKER CLUSTER
POSTGRESQL
CACHE
MESSAGE BROKER
OBJECT STORAGE
OBSERVABILITY
BACKUP SYSTEM
```

Each component shall be independently monitored.

---

39. DEVELOPMENT TOPOLOGY

Local development shall provide a reproducible environment.

Conceptually:

```
docker compose
    ↓
API
POSTGRES
CACHE
QUEUE
OBJECT STORAGE
WORKER
```

Developers should be able to initialize the infrastructure without manually configuring every dependency.

---

40. TEST ENVIRONMENT TOPOLOGY

Automated testing shall execute against isolated infrastructure.

The test environment shall not use production:

```
database
storage
queue
credentials
AI provider credentials
```

Test fixtures shall be deterministic where practical.

---

41. STAGING ENVIRONMENT

Staging shall approximate production architecture.

It shall be used for:

```
release validation
migration testing
integration testing
performance testing
deployment verification
end-to-end testing
```

A production deployment should originate from an artifact that has successfully passed staging gates.

---

42. PRODUCTION DEPLOYMENT

Production deployment shall be controlled through the release pipeline.

The deployment process shall:

```
validate artifact
validate configuration
validate migrations
deploy services
run health checks
verify dependencies
monitor startup
```

Failed deployments shall halt or roll back according to deployment policy.

---

43. ZERO-DOWNTIME DEPLOYMENT

Where practical, OneDraft services shall support rolling or blue-green deployment.

The API contract shall remain compatible during transitional deployment states.

Database migrations shall therefore follow backward-compatible migration patterns where possible.

---

44. MIGRATION SAFETY

Destructive database changes shall use staged migrations.

Preferred pattern:

```
ADD
 ↓
MIGRATE
 ↓
VERIFY
 ↓
SWITCH
 ↓
REMOVE
```

A column or table should not be removed in the same deployment that introduces its replacement when active services may still depend on it.

---

45. DEPENDENCY FAILURE

Services shall tolerate temporary dependency failures.

Examples:

```
DATABASE UNAVAILABLE
QUEUE UNAVAILABLE
OBJECT STORAGE UNAVAILABLE
AI PROVIDER UNAVAILABLE
EVENT BUS UNAVAILABLE
```

The service shall fail predictably rather than corrupting project state.

---

46. RETRY POLICY

Retries shall be bounded.

The system shall use:

```
maximum attempts
exponential backoff
jitter
dead-letter handling
```

Non-idempotent operations shall not be blindly retried.

---

47. TIMEOUT POLICY

All network and worker operations shall have explicit timeouts.

Examples:

```
API request timeout
database query timeout
AI request timeout
geometry job timeout
document generation timeout
storage operation timeout
event publication timeout
```

Timeouts shall prevent indefinite resource occupation.

---

48. GRACEFUL SHUTDOWN

Services shall support graceful shutdown.

A worker receiving termination shall:

```
stop accepting new work
finish or safely release current work
acknowledge completed jobs
persist required state
close connections
exit
```

The infrastructure must avoid terminating active work without a recovery mechanism.

---

49. DEPLOYMENT IDENTIFICATION

Every running service shall expose:

```
application version
build identifier
source commit
environment
deployment timestamp
```

This information shall be available to observability systems.

---

50. INFRASTRUCTURE AS CODE

Production infrastructure shall eventually be represented as code.

The repository shall contain infrastructure definitions for:

```
networking
compute
database
storage
queues
secrets
monitoring
DNS
TLS
deployment
```

Manual production infrastructure changes should be minimized.

---

51. INFRASTRUCTURE REPOSITORY STRUCTURE

Conceptually:

```
infrastructure/
├── docker/
├── compose/
├── environments/
│   ├── development/
│   ├── test/
│   ├── staging/
│   └── production/
├── networking/
├── database/
├── storage/
├── queues/
├── observability/
└── deployment/
```

The exact implementation may use Terraform, Kubernetes manifests, Helm, or another infrastructure platform as the deployment architecture matures.

---

52. DOCKER COMPOSE DEVELOPMENT STACK

The initial local infrastructure should support:

```
postgres
redis
object-storage
api
worker
```

Optional services:

```
event-bus
observability
geometry-worker
documentation-worker
```

The local stack must remain simple enough for rapid development.

---

53. PRODUCTION ORCHESTRATION

The production platform may use:

```
Kubernetes
managed container orchestration
or equivalent infrastructure
```

The choice shall be driven by operational requirements rather than architectural fashion.

The application contracts must remain platform-independent.

---

54. DATABASE STORAGE VOLUME

PostgreSQL data shall reside on persistent storage.

Container replacement must not destroy database state.

Database storage must therefore be external to the ephemeral container filesystem.

---

55. WORKER STORAGE

Workers shall treat local disk as temporary workspace.

Temporary geometry, rendering, conversion, and extraction files shall be removed after job completion unless explicitly persisted as artifacts.

---

56. ARTIFACT STORAGE SECURITY

Object storage shall enforce:

```
private buckets
authenticated access
least-privilege service credentials
server-side encryption where supported
object versioning where appropriate
audit logging
```

Public artifact exposure shall require an explicit controlled mechanism.

---

57. BACKUP INFRASTRUCTURE

Infrastructure shall support:

```
PostgreSQL backups
object storage replication
configuration backup
infrastructure definitions
critical event preservation
```

Backup systems shall be monitored for successful completion.

---

58. RECOVERY INFRASTRUCTURE

Recovery shall be tested against:

```
database failure
worker failure
API failure
storage failure
queue failure
deployment failure
regional infrastructure failure
```

The objective is not merely to possess backups but to demonstrate that they can restore operational state.

---

59. PRODUCTION SECRETS ROTATION

Credentials shall support controlled rotation.

Rotation shall not require uncontrolled application modification.

Where possible:

```
issue new credential
deploy updated configuration
verify operation
revoke old credential
```

---

60. INFRASTRUCTURE OBSERVABILITY

Infrastructure shall integrate with the OneDraft observability layer.

Metrics shall include:

```
CPU
memory
disk
network
database connections
queue depth
worker utilization
storage utilization
request rate
latency
error rate
```

---

61. INFRASTRUCTURE LOGGING

Infrastructure logs shall be centralized where operationally appropriate.

Logs shall include:

```
timestamp
service
environment
instance
severity
request_id
job_id
project_id where permitted
message
```

Sensitive credentials and confidential payloads must not be logged.

---

62. INFRASTRUCTURE TRACING

Distributed tracing shall permit a request to be followed across:

```
API
domain service
database
queue
worker
artifact storage
event bus
```

Trace propagation shall use a standard correlation mechanism.

---

63. RESOURCE GOVERNANCE

The platform shall monitor infrastructure consumption by:

```
organization
project
service
worker class
job type
environment
```

This provides the foundation for future billing and entitlement metering.

---

64. COMPUTATIONAL WORKLOAD ISOLATION

Heavy computational workloads shall not be permitted to starve ordinary API operations.

For example:

```
GEOMETRY JOB
```

shall not consume all resources required for:

```
PROJECT READ
PROJECT WRITE
ARIA REQUEST
REDLINE REVIEW
```

Resource classes and worker pools shall enforce this separation.

---

65. SECURITY BOUNDARY

The infrastructure shall enforce the security boundaries defined by the Identity, Access Control & Security System.

Infrastructure credentials must not bypass application authorization.

A database administrator capability shall remain distinct from ordinary project-user permissions.

---

66. ADMINISTRATIVE ACCESS

Production administrative access shall be restricted.

Administrative infrastructure access should require:

```
authenticated identity
authorized role
auditable access
controlled network path
```

Administrative credentials shall not be embedded in application containers.

---

67. PRODUCTION CHANGE CONTROL

Infrastructure changes shall pass through controlled change management.

Conceptually:

```
CHANGE
  ↓
CODE
  ↓
REVIEW
  ↓
TEST
  ↓
STAGING
  ↓
DEPLOY
  ↓
VERIFY
```

Emergency procedures may bypass ordinary sequencing only under documented operational policy.

---

68. DEPLOYMENT ARTIFACT PROMOTION

The preferred promotion path is:

```
BUILD
  ↓
TEST
  ↓
STAGING
  ↓
APPROVAL
  ↓
PRODUCTION
```

The same immutable artifact should be promoted rather than rebuilt independently for production.

---

69. VERSION COMPATIBILITY

Infrastructure versions must remain compatible with:

```
database schema
API version
worker version
event schemas
artifact formats
domain model version
```

Compatibility checks shall be part of release validation.

---

70. INFRASTRUCTURE FAILURE MODEL

OneDraft infrastructure shall assume that components can fail.

The architecture shall therefore support:

```
restart
retry
replacement
rescheduling
recovery
reconciliation
```

No single ephemeral compute instance shall constitute the sole source of project state.

---

71. STATEFUL VS STATELESS COMPONENTS

Stateful:

```
PostgreSQL
object storage
persistent event history
```

Potentially stateful infrastructure:

```
message broker
cache
```

Stateless:

```
API
domain services
Aria orchestration
workers
document service
validation service
```

This distinction governs scaling and recovery strategy.

---

72. PRODUCTION TOPOLOGY PRINCIPLE

The production topology shall preserve:

```
AUTHORITATIVE STATE
        ↓
TRANSACTIONAL SERVICES
        ↓
ASYNC COMPUTATION
        ↓
DERIVED ARTIFACTS
```

Derived artifacts must never become the authoritative project model.

---

73. INFRASTRUCTURE / DOMAIN SEPARATION

The infrastructure subsystem shall not determine:

```
architectural validity
building-code compliance
project acceptance
```

```
design intent
model semantics
```

Those responsibilities belong to the domain and application layers.

Infrastructure provides the execution substrate.

---

74. DEPLOYMENT / DOCUMENT PROVENANCE

Final document packages shall be traceable to:

```
software version
model version
revision
code manifest
validation run
generation job
artifact hash
```

This provides infrastructure-level support for OneDraft document provenance.

---

75. INFRASTRUCTURE RECONSTRUCTION TEST

A complete environment must eventually be reconstructable from:

```
source repository
container images
database migrations
infrastructure definitions
configuration
secrets
backups
artifact storage
```

A deployment that depends on undocumented manual infrastructure configuration is considered incomplete.

---

76. MVP INFRASTRUCTURE SUCCESS CRITERIA

The infrastructure implementation is successful when it can:

Start the complete local OneDraft stack

Initialize PostgreSQL

Run database migrations

Start the API

Start workers

Process queued jobs

Store artifacts

Publish and consume events

Execute geometry workloads

Execute validation workloads

Generate documentation

Persist project state

Restart services without losing authoritative state

Run isolated test environments

Deploy a reproducible staging environment

---

77. PRODUCTION SUCCESS CRITERIA

Production infrastructure is successful when:

```
API
 ↓
DOMAIN
 ↓
DATABASE
 ↓
QUEUE
 ↓
WORKERS
 ↓
ARTIFACT STORAGE
 ↓
EVENT BUS
```

operates as a coherent platform with:

```
authentication
authorization
monitoring
logging
backup
recovery
scaling
deployment control
```

and no critical component depends upon undocumented manual intervention.

---

78. FIRST INFRASTRUCTURE INTEGRATION TEST

The first complete infrastructure test shall execute:

```
1. Build application images

2. Start infrastructure

3. Initialize PostgreSQL

4. Apply migrations

5. Start API

6. Start workers

7. Create organization

8. Create project

9. Submit model operation

10. Queue geometry job

11. Execute worker

12. Persist geometry artifact

13. Publish completion event

14. Execute validation job

15. Generate drawing

16. Store drawing artifact

17. Generate document package

18. Store final artifact
```

```
19. Retrieve artifact

20. Verify SHA-256 hash

21. Restart worker

22. Verify project state remains intact

23. Restart API

24. Verify API reconnects to persistence

25. Retrieve complete project history
```

This becomes the first infrastructure-level deployment regression test.

---

79. INITIAL INFRASTRUCTURE ARTIFACTS

The next implementation package shall contain:

```
docker-compose.yml
Dockerfiles
environment templates
database deployment configuration
object-storage configuration
queue configuration
worker deployment configuration
health-check configuration
infrastructure documentation
```

Production infrastructure definitions shall be introduced as deployment requirements mature.

---

80. RELATIONSHIP TO PRECEDING SYSTEMS

Infrastructure shall consume and support the established OneDraft systems:

```
Project Model
Geometry Engine
Compliance Engine
Validation Engine
Documentation Engine
Review & Acceptance
Runtime Orchestrator
Job & Worker System
Artifact Storage
Event Bus
```

```
Identity & Security
Observability
Testing
CI/CD
```

It shall not redefine those systems.

---

81. INFRASTRUCTURE DEPENDENCY GRAPH

The resulting infrastructure relationship is:

```
IDENTITY
   ↓
API GATEWAY
   ↓
APPLICATION SERVICES
   ↓
DOMAIN SERVICES
   ↓
POSTGRESQL
   ↓
EVENT BUS
   ↓
JOB QUEUE
   ↓
WORKER POOLS
   ↓
ARTIFACT STORAGE
```

Cross-cutting infrastructure:

```
CONFIGURATION
SECRETS
OBSERVABILITY
BACKUP
NETWORKING
SECURITY
CI/CD
```

---

82. DEPLOYMENT INVARIANT

The following invariant shall be maintained:

```
NO DEPLOYMENT
        ↓
WITHOUT
        ↓
COMPATIBLE APPLICATION
```

```
        +
COMPATIBLE DATABASE
        +
COMPATIBLE WORKERS
        +
COMPATIBLE EVENTS
        +
COMPATIBLE ARTIFACT CONTRACTS
```

This prevents independently upgraded components from silently corrupting the project execution environment.

---

83. INFRASTRUCTURE PRINCIPLE

OneDraft infrastructure shall be:

Reproducible

Observable

Recoverable

Scalable

Secure

Versioned

Automatable

Environment-separated

Failure-tolerant

Domain-independent

---

84. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now includes:

Foundational Technical Specification

System Architecture

MVP Technical Stack

API & Domain Schema

Database & OpenAPI Contract

Executable Foundation

Application Runtime

Core Application Services

Project Model

Geometry Engine

Compliance Engine

Validation Engine

Documentation Engine

Review & Acceptance

Runtime Orchestration

Job & Worker Execution

Artifact & Storage Management

Integration & Event Bus

Identity, Access & Security

Observability

Testing & Quality Assurance

CI/CD

Infrastructure & Deployment Architecture

The infrastructure layer now provides the executable environment in which these systems operate.

---

85. NEXT IMPLEMENTATION ARTIFACTS

The immediate infrastructure implementation package shall consist of:

```
docker-compose.yml
Dockerfile.api
Dockerfile.worker
Dockerfile.geometry-worker
Dockerfile.documentation-worker
Dockerfile.validation-worker
.env.example
infrastructure/
deployment/
```

The exact deployment provider shall remain abstracted until production capacity requirements justify the final infrastructure platform.

86. FINAL ARCHITECTURAL DECISION

OneDraft shall not depend upon a single machine, container, worker, or deployment instance.

The authoritative architecture remains:

```
PROJECT MODEL
      ↓
APPLICATION SERVICES
      ↓
TRANSACTIONAL STATE
      ↓
ASYNC COMPUTATION
      ↓
DERIVED ARTIFACTS
```

Infrastructure provides the execution substrate for this architecture.

The infrastructure layer therefore exists to make the OneDraft computational system **reproducible, scalable, observable, recoverable, and deployable** without becoming the system of record itself.